



Programming Project

MATB32: Linear Algebra

J. K. Gunnarsson, C. A. Santillan Moreno, V. Schüller, N. Kotarsky

This assignment has 2 tasks.

The goal of this project is to gain a better understanding of theoretical concepts with the help of practical implementations **in Python**. Note that each corresponding task may contain several subtasks. In some tasks you are asked to do theoretical derivations, in others you are asked to implement numerical methods and visualize results. For the provided tasks we recommend using `numpy`, `scipy` and `matplotlib` for numerical computations and plotting. You are free to use any additional libraries you find useful, but make sure to mention them in your submission. You are also free to use any numerical method you find suitable to solve the tasks, but make sure to justify your choice. You should submit a single Jupyter notebook file `.ipynb` or Python script `.py` containing your code and plots. You should also include a short report in PDF¹ format explaining your derivations, results and findings. This includes adding every plot as a figure, and including brief discussions regarding your results and interpretations. Other than that, you may structure the report however you wish, as long as it is clear which parts (discussion and figures) correspond to which tasks.

Task 1 – Linear Regression, Projections & Reflections, Least Squares

Assume we have a series of measurements $\mathbf{y} = (y_1, \dots, y_n)^T \in \mathbb{R}^n$ given at points $\mathbf{x} = (x_1, \dots, x_n)^T \in \mathbb{R}^n$. We want to approximate the data points by a linear function, a process called *(simple) linear regression*. That is, we aim to find $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ such that (Treil, 4.3):

$$f(\mathbf{x}) = a\mathbf{x} + b =: \hat{\mathbf{y}} \approx \mathbf{y}, \quad (a, b) = \arg \min_{a, b \in \mathbb{R} \times \mathbb{R}} \|a\mathbf{x} + b - \mathbf{y}\|^2.$$

We want to find coefficients a, b in a way that optimally approximates $\mathbf{y}$ as a linear fit given our data $\mathbf{x}$.

In this task, you are going to work with **four** datasets, **one** in `regression_1.dat` and **three** in `regression_2.dat`. These can be downloaded on Canvas. We note that `regression_2.dat` contains one dataset for $\mathbf{x}$ and **three** different

¹The standard at our department is to use L^AT_EX. If you are not familiar with L^AT_EX, please let the project responsible know!

datasets for $\mathbf{y}$, i.e., three different measurement series for the same $\mathbf{x}$ values. Your submitted code should show results for **all four cases** in individual plots.²

1. Express each data set as a least squares problem and find the solution for a, b (Elfstrom, Ex. 3.40 and Trel, 4.3). Provide the mathematical least squares formulation (Elfstrom, 3.5) for this specific problem. Solve the minimization problem directly with `scipy.optimize.fmin`, and also as a least squares problem with `numpy` (for instance, `numpy.linalg.lstsq` or solving the equation system directly using (Elfstrom, 3.5)).
2. Plot the resulting linear functions $f(\mathbf{x})$ and data points for **every** dataset. Add the plots in your report and comment on the quality of the fit. Is it a good fit, or would you need to have higher order polynomial fits (i.e. quadratic)? Would you be able to remove any data points to improve the fit? If so, which ones and why?
3. Create a surface plot with `matplotlib` of the error $\epsilon(a, b) = \|\hat{\mathbf{y}}(a, b) - \mathbf{y}\|$ for different values of a, b (Treil, 4.1). In the plot, mark the values you found for a, b in the first subtask. Given the surface plots, can you graphically justify that your solution (a, b) corresponds to a unique solutions? What is the shape of the error surface? Include the plot and discussion in your report.
4. Provide a brief proof that the solution (a, b) you found in the previous subtask is indeed a *unique* minimizer of the error (Treil, 4.1). It might be helpful to note that with $\hat{\mathbf{A}} = [\mathbf{x}, \mathbf{1}] \in \mathbb{R}^{n \times 2}$ and $\hat{\mathbf{x}} = [a, b]^T \in \mathbb{R}^2$, the linear regression problem can be written as

$$\beta = \arg \min_{\beta \in \mathbb{R}^2} \|\hat{\mathbf{A}}\hat{\mathbf{x}} - \mathbf{y}\|^2.$$

Linear regression can be viewed as a projection. This assertion is discussed in the literature (Elfstrom, 3.5 and Treil, 4.2).

5. Provide a brief proof that the least squares solution $\hat{\mathbf{y}}$ is indeed a projection of $\mathbf{y}$. To do this, find a matrix $\mathbf{P}$ satisfying $\hat{\mathbf{y}} = \mathbf{P}\mathbf{y}$ for each separate solution and data set. Provide the theoretical motivation for your construction of $\mathbf{P}$ (Elfstrom, 5.3).
6. Show numerically that $\mathbf{P}$ is a projection matrix with $\mathbf{P}^2 = \mathbf{P}$ with `allclose` in `numpy`. Is it an *orthogonal* projection? Construct a brief function in Python which checks if a matrix $\mathbf{P}$ is a projection and one to check if it is also an orthogonal projection with $\mathbf{P} = \mathbf{P}^T$ (Elfstrom, Thm 5.27).
7. Extend your first plot with the new data points $\hat{\mathbf{y}} = \mathbf{P}\mathbf{y}$. What does it represent? Also, provide the new plots in your report.
8. Using only $\hat{\mathbf{y}}$ and $\mathbf{y}$, numerically verify that $\hat{\mathbf{y}}$ is the result of an orthogonal projection (Elfstrom, Def 5.24).

²It can be helpful to first solve the exercises for the data in `regression_1.dat` before re-working the code to solve the tasks for all four data sets.

- Define the reflection matrix $\mathbf{R} = 2\mathbf{P} - \mathbf{I}$ associated with $\mathbf{P}$. Extend the plot to show the reflected vector $\mathbf{R}\mathbf{y}$. Provide a mathematical justification for your formulation and also a short code snippet to test if a matrix $\mathbf{R}$ corresponds to a reflection matrix. Is this result expected from (Elfstrom, Thm. 5.27)?

Task 2 – Eigenvalues, Eigenvectors, Recurrence Relations

In this task, we work with the following recurrence relation:

$$\begin{bmatrix} a_{n+1} \\ b_{n+1} \\ c_{n+1} \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 3 & 2 \\ -3 & 4 & 3 \\ 2 & 3 & 1 \end{bmatrix}}_{=: \mathbf{A}} \begin{bmatrix} a_n \\ b_n \\ c_n \end{bmatrix}.$$

We denote the initial values $[a_0, b_0, c_0]^T$ as $\mathbf{z}_0$. Perform the following tasks for $\mathbf{z}_0 = [9, 1, 9]^T$, $\mathbf{z}_0 = \alpha[1, 0, 1]^T$, and $\mathbf{z}_0 = \alpha[1, 12, -19]^T$ for $\alpha \in \{1, \frac{1}{19}\}$. It might be helpful to write a `list` which stores these initial values for each α and loop over them in your code.

- Derive a general closed form expression for $\mathbf{z}_n$ in terms of the eigenvalues and eigenvectors of $\mathbf{A}$ (Elfstrom, 6.4). Here you can use `scipy.eig` given that $\mathbf{A}$ is an `numpy.ndarray` or similar datatype. You may note that you can diagonalize $\mathbf{A}$. Given an arbitrary initial value $\mathbf{z}_0$, under which conditions on $\mathbf{z}_0$ with the eigenvectors and eigenvalues $\mathbf{A}$ do the iterates $\mathbf{z}_n$ converge for $n \rightarrow \infty$? Document the eigenvalues and eigenvectors you found for $\mathbf{A}$ and how to express $\mathbf{z}_0$ in terms of them (Elfstrom, Thm 6.15 & Col 6.16).
- Construct a function in Python which computes the iterates $\mathbf{z}_n$ for $n = 0, 1, 2, \dots, N$ **iteratively** without using the previous eigenvectors for a given initial value $\mathbf{z}_0$, a given number of iterations N and an input matrix $\mathbf{A}$. Represent $\mathbf{z}_n$ and $\mathbf{A}$ in the datatype `numpy.ndarray` using, for instance, `numpy.array`. Write a loop which run this function for each provided initial values $\mathbf{z}_0$ and for each α up-to $N = 400$ iterations. Document your results, if the limit of $\mathbf{z}_n$ only switch sign for sufficiently large N ³, does the directional vector of $\mathbf{z}_n$ converge? If so, to what? For all cases write down the initial value $\mathbf{z}_0$ and α and what directional vector converges to. Does this agree with your theoretical findings from the previous subtask? See what the limit would be if you did not iterate but instead just used your theoretical result from above. Do they agree? If not, why?⁴

Consider now the sequence of normalized iterates:

$$\mathbf{v}_0 = \frac{\mathbf{z}_0}{\|\mathbf{z}_0\|}, \quad \mathbf{v}_n = \frac{\mathbf{A}\mathbf{v}_{n-1}}{\|\mathbf{A}\mathbf{v}_{n-1}\|}, \quad \text{for } n > 0.$$

³In the sense that for instance at $N = 399$ then $\mathbf{z}_{399} = -\mathbf{z}_{398}$

⁴You are supposed to get results which do not exactly agree, so don't worry!

3. Give a brief theoretical motivation to why the sequence $\mathbf{v}_n$ has a convergent directional vector independently of the initial value $\mathbf{z}_0 \neq \mathbf{0}$.⁵ What is the limit $\mathbf{v}$ in terms of the eigenvalues and eigenvectors of $\mathbf{A}$ given an initial $\mathbf{z}_0$? Is it unique? You do not need to provide a full proof, but outline the main arguments.
4. Implement a function in Python which computes the normalized iterates $\mathbf{v}_n$ for $n = 0, 1, 2, \dots, N$ **iteratively** for a given initial value $\mathbf{z}_0$, a given number of iterations N and an input matrix $\mathbf{A}$. Write a loop which runs this function for each provided initial values $\mathbf{z}_0$ and for each α up to $N = 400$. Report your results. Does this agree with your theoretical findings from the previous subtask and your conclusions from the previous task?
5. Implement in Python a method which for every normalized iterate also computes the quantity $q_n = \mathbf{v}_n^T \mathbf{A} \mathbf{v}_n$. Determine the limit q . What is the relation of q to the eigenvalues of $\mathbf{A}$ and the limit $\mathbf{v}$ for each $\mathbf{z}_0$?

Now, analyse the convergence of the sequences $\mathbf{v}_n$ and q_n compared to their corresponding limit points $\mathbf{v}^*$ and q^* . Take both the theoretical limit points $\mathbf{v}$ and q , and also study the following for the numerical ones you obtain $\mathbf{v}_{400}$ and q_{400} , if the results differ. We stress that you should use the same initial values $\mathbf{z}_0$ as before.

6. Motivate that it is not sufficient to have $\|\mathbf{v}_n - \mathbf{v}_{n-1}\| < \varepsilon$ or $\|\mathbf{v}_{n+1} - \mathbf{v}_{n-1}\| < \varepsilon$ for some small $\varepsilon > 0$ to ensure that $\mathbf{v}_n$ is close to the limit $\mathbf{v}$. Provide the justification by solving the simpler problem: does the sequence defined by $x_n = \sum_{j=1}^n \frac{1}{j}$ for $n = 1, 2, \dots$ satisfy $|x_n - x_{n-1}| < \varepsilon$ or $|x_{n+1} - x_{n-1}| < \varepsilon$ for some small $\varepsilon > 0$ dependent on some $n > N$? Does x_n converge? Provide a brief justification using your knowledge from single variable analysis.
7. How many iterates do you need to fulfil $\|\mathbf{v}_n - \mathbf{v}^*\| < \varepsilon$ for $\varepsilon = 10^{-8}$? Make a table reporting the number of iterates n when running against the theoretical limit $\mathbf{v}^* = \mathbf{v}$ and the numerical limit $\mathbf{v}^* = \mathbf{v}_{400}$.
8. Repeat the previous task by varying ε between 0.1 and 10^{-14} , plot the number of iterates versus ε in a semi-log plot using `matplotlib.pyplot.semilogy`. Do the same for q_n and its limit q^* , also for the theoretical limit $q^* = q$ and numerical limit $q^* = q_{400}$. If $q_{400} \neq q$ include both in the plot. Why do they differ? What can you say about the iteration count n with respect to ε until convergence?

⁵Hint: What is $\|\mathbf{v}_n\|$?